

Inbound Attendance Processing

Asynchronous ingestion of external faculty attendance data

FINKI — Faculty of Computer Science and Engineering

Reference document: explains the queue-style design and the reasoning behind each component

What problem are we solving?

The requirement: different faculties run their own attendance systems and need to send attendance data into ours. When a student attends a consultation, a faculty system sends a payload like the one below to our API.

```
{
  "userId": "123",
  "consultationId": "456",
  "attendedAt": "2026-06-16T10:00:00",
  "notes": "Student attended"
}
```

THE BEGINNER APPROACH (WRONG)

```
Receive request
↓
Immediately create Attendance
```

WHAT THE REQUIREMENT ACTUALLY WANTS

```
Receive request
↓
Save request
↓
Mark as Pending
↓
Process later
```

Why: requests might arrive one at a time, or in batches of 100 or 10,000. External systems shouldn't sit waiting while ours does all the work synchronously — so a queue-like approach is used instead.

Building the Pipeline

STEP-BY-STEP

STEP 1 InboundAttendanceRequest

InboundAttendanceRequest.cs

```
public record InboundAttendanceRequest(
    string UserId,
    string ConsultationId,
    DateTime AttendedAt,
    string? Notes
);
```

Why: ASP.NET receives JSON as text, but C# works with objects. This record defines the shape that incoming JSON is automatically converted into: JSON → InboundAttendanceRequest.

STEP 2 InboundAttendanceEntry

InboundAttendanceEntry.cs — one of the most important files

Not the same as Attendance. Think of a package delivery company: when a package arrives, a tracking record ("Tracking Number: 123, Status: Pending") is created first, before the package is actually delivered. This entity is exactly that tracking record.

Field	Purpose
<code>RawPayload</code>	Stores the original JSON exactly as received, so Quartz can read it later.
<code>Status</code>	Where the request currently is: Pending, Processing, Completed, or Failed.
<code>ApiClientId</code>	Which faculty sent this request.
<code>ReceivedAt</code>	When the request arrived.
<code>ProcessedAt</code>	When processing finished.
<code>ErrorMessage</code>	Failure reason, e.g. "Consultation not found".
<code>CreatedAttendanceId</code>	ID of the Attendance record created from this entry.

Status	Meaning
Pending	Request arrived, not processed yet.
Processing	Quartz is currently working on it.
Completed	Attendance successfully created.
Failed	Something went wrong.

STEP 3 DbContext registration

ApplicationDbContext

```
public DbSet<InboundAttendanceEntry>
    InboundAttendanceEntries { get; set; }
```

Why: Entity Framework only creates tables for entities it knows about. Without this DbSet, the InboundAttendanceEntries table would never exist; after migration, EF creates it in SQL Server.

STEP 4 IInboundAttendanceService — the contract

The question this interface answers: what operations do we need on InboundAttendanceEntries?

- Create
- Get
- Get Pending
- Update

STEP 5 InboundAttendanceService — talking to the database

CREATEASYNC

Given a POST payload like `{"userId": "123"}`, this does **not** create an Attendance. It creates an InboundAttendanceEntry with `Status = Pending` and saves it.

```
Id: 1 Status: Pending
```

No Attendance exists yet.

GETBYIDASYNC

Used by the GET /status endpoint to find one specific request.

GETPENDINGBATCHASYNC

The requirement says "process 5 requests" — so `GetPendingBatchAsync(5)` returns only Pending entries, and only 5 records.

UPDATEASYNC

Called whenever status changes: Pending → Processing → Completed, or Pending → Failed.

STEP 6 ExternalAttendanceController — the door to the system

External faculties never call services directly; they call the controller.

POST /API/EXTERNAL/ATTENDANCE

1. Controller receives the request.
2. Validates the API key — the requirement states only valid keys can use the API, checked via the `X-API-KEY` header.
3. Serializes the request object back to a JSON string, so the original payload can be stored.
4. Calls `CreateAsync`, which creates an `InboundAttendanceEntry` with status Pending.
5. Returns **202 Accepted**.

```
{ "id": "ABC123" }
```

Attendance still does not exist yet.

GET /API/EXTERNAL/ATTENDANCE/{ID}/STATUS

External faculty asks "has my request been processed?" The controller reads the entry and returns Pending, Completed, or Failed.

STEP 7 InboundAttendanceProcessor

Controller saves requests. Processor creates Attendances. These are different responsibilities.

PROCESSPENDINGATTENDANCEASYNC

Every Quartz run calls `GetPendingBatchAsync(5)`, which returns the oldest 5 Pending entries. If the database holds 7 pending entries, only the first 5 are taken in this run.

PROCESSATTENDANCEENTRY

1. **Deserialize** — the stored JSON is converted back into an `InboundAttendanceRequest`.
2. **Find Consultation** — verified via `_consultationService`. Not found → Failed. Found → continue.
3. **Create AttendanceDto** — builds the DTO required by `AttendanceService`.
4. **Create Attendance** — `_attendanceService.CreateAsync(...)` inserts the actual Attendance record.
5. **Save Attendance ID** — stored in `entry.CreatedAttendanceId`.

STEP 8 QuartzAttendanceJob

Quartz is just a scheduler — an alarm clock for the application — that wakes up every 30 seconds and calls `_processor.ProcessPendingAttendanceAsync()`. It has no knowledge of JSON, Attendance, or Consultation; it only tells the processor to run.

Complete Real-Life Example

Faculty sends:

```
{
  "userId": "123",
  "consultationId": "456",
  "notes": "Present"
}
```

1. Controller receives it and creates an InboundAttendanceEntry.

```
Id: A1 Status: Pending
```

2. Controller returns:

```
{ "id": "A1" }
```

3. 30 seconds later, Quartz runs and the processor loads entry A1.

4. Status changes: Pending → Processing.

5. JSON is read, the Consultation is found, and Attendance is created.

6. The entry stores `CreatedAttendanceId = ATT001`.

7. Status changes: Processing → Completed.

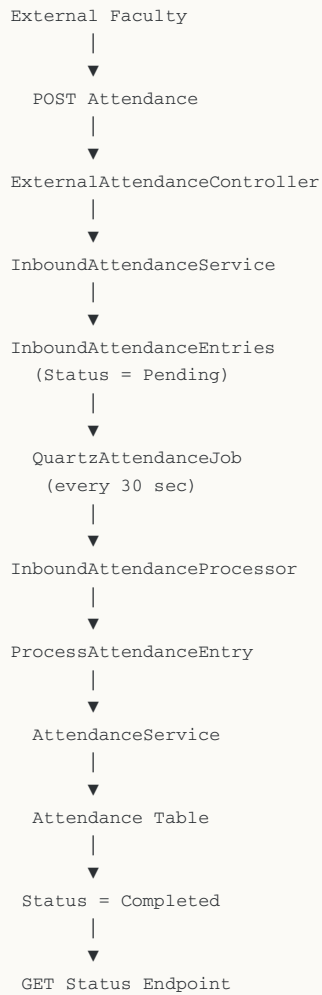
External faculty later calls:

```
GET /api/external/attendance/A1/status
```

Response:

```
{ "status": "Completed", "createdAttendanceId": "ATT001" }
```

Final Architecture



Explained this way, the goal isn't just describing code — it's explaining the entire architecture and the reason each component exists, which is exactly what holds up under exam or presentation questioning.